

EE366200 DSP Laboratory

Lab 8 Seam Carving for Content-Aware Image Resizing

111061220 林進成

1. Introduction

Lab 8 explores content-aware image resizing using seam carving, a method that changes image dimensions by following paths of minimal visual significance rather than uniformly scaling or cropping. The lab motivates the approach by defining pixel importance with an energy function derived from image gradients so that strong edges and textured regions are preserved while low-contrast areas are preferentially altered. Students practice identifying optimal vertical and horizontal seams through dynamic programming on cumulative energy maps and then modify images by removing or inserting these seams to achieve targeted size changes. Beyond basic resizing, the lab frames seam carving as a tool for structure-preserving editing, setting up later work on enlarging images by seam insertion and steering seams to pass through specified regions for object removal. Throughout, the emphasis is on understanding how energy design and optimal seam search interact to protect salient content while adapting aspect ratio, providing a practical foundation for content-aware image manipulation, with comparative evaluations, boundary cases, and artifacts discussed to guide implementation choices.

2. Objectives

The in-class demonstration aims to validate content-aware image resizing using seam carving in two complementary settings. First, the demonstration must reduce an image by repeatedly removing optimal low-energy seams so that salient edges and structures remain visually intact after the size change. Second, it must enlarge the image by inserting seams derived from previously computed optimal paths, increasing the target dimension while preserving coherence along object boundaries and textured regions. The deliverable is a pair of results that correspond to a faithful reduction and a faithful enlargement of the same source image, shown alongside the original to illustrate that important content is preferentially preserved rather than uniformly distorted.

3. Approach

The procedure begins with the original image (Fig. 1).



Fig. 1 The original image

An energy function based on image gradients is computed so that textured regions and edges attain higher energy and homogeneous areas attain lower energy. Using this energy map, a cumulative cost table is constructed via dynamic programming in the chosen direction (vertical or horizontal), and an admissible path that connects opposite borders with minimal accumulated energy is recovered by backtracking. Removing the pixels along this path produces an image that is reduced by one column (for a vertical seam) or one row (for a horizontal seam). Repeating the cycle of energy computation, cumulative cost propagation, seam backtracking, and seam removal yields the reduced result (Fig. 2), where the removal count equals the net change in dimension.



Fig. 2 The reduced image

For enlargement, the method first identifies a set of k optimal seams by running the same path-finding process on a working copy of the image and recording each recovered path. Because each removal changes pixel coordinates, the recorded paths

are updated so that their indices remain consistent with the original coordinate system used for insertion. Seam insertion then proceeds on the original image by adding one pixel along each recorded path and assigning intensities through local interpolation or neighbor averaging to minimize visible artifacts. Iterating this operation across all k paths increase the image width (or height) by k and produces the enlarged result (Fig. 3), which serves as the counterpart to the reduction while maintaining structural continuity across inserted seams.



Fig. 3 The enlarged image

Together, these steps constitute a reproducible pipeline for in-class demonstration: compute energy, select seams, and remove or insert them with proper index bookkeeping and interpolation, yielding reduced and enlarged images (Figs. 2–3) from the unchanged original (Fig. 1).

4. Discussion

I.

In the process of seam insertion, what will happen if we insert the seam with minimum energy directly. Is it better than what we did in the demo section? Show your results and give some explanations.

Ans:

To assess whether inserting the minimum-energy seam directly is preferable to the demo method, I enlarged the image by a factor of two in width. The demo result, obtained by recording k seams on a working copy and inserting them into the original with neighbor interpolation, is shown in Fig. 4. The alternative result, which inserts the currently optimal seam at each step without interpolation, is shown in Fig. 5.

In Fig. 5, repeated selection of the same low-energy corridor produces visible vertical banding and duplicated textures across the ocean and shoreline, together with faint ghost edges near high-contrast boundaries. By contrast, Fig. 4 exhibits smoother transitions: interpolation blends newly inserted pixels with their neighbors and pre-recording a diverse set of seams prevents over-concentration in any single region. Consequently, the demo pipeline yields higher visual fidelity than direct, non-interpolated insertion.



Fig. 4 Inserting seams with neighbor interpolation



Fig. 5 Inserting seams with minimum energy directly

In Fig. 5, repeated selection of the same low-energy corridor produces visible vertical banding and duplicated textures across the ocean and shoreline, together with faint ghost edges near high-contrast boundaries. By contrast, Fig. 4 exhibits smoother transitions: interpolation blends newly inserted pixels with their neighbors and pre-recording a diverse set of seams prevents over-concentration in any single region. Consequently, the demo pipeline yields higher visual fidelity than direct, non-interpolated insertion.

When the enlargement amount equals the original width (doubling), the non-interpolation variant effectively degenerates to nearest-neighbor horizontal scaling.

Each insertion duplicates existing pixel columns along the selected seam locations; after W insertions on an image of width W , the output consists of piecewise column repetitions. This is precisely the discrete operation performed by nearest-neighbor upsampling by a factor of two: every source column is copied to one or more destination columns with no new values created between them. Interpolation in the demo method, however, synthesizes intermediate intensities between adjacent columns, which corresponds to a true resampling step rather than simple replication. The difference is visible by comparing Fig. 5 (column replication and banding) with Fig. 4 (blended boundaries and reduced artifacts).

II.

Finish the horizontal seams part in “reduceImageByIndexArray.py” and “findOptSeam.py”. What if we remove horizontal seams from the image sea.jpg. Is there any problem?

Ans:

I completed the horizontal pathway by adding a left-to-right dynamic program in “findOptSeam.py” for `seamDirection == 1`, which accumulates energy across columns and backtracks from the minimum in the last column to recover an admissible row at each column. In “reduceImageByIndexArray.py”, those indices are applied row-wise so that one pixel per column is removed at the recovered row, reducing the height

by one for each iteration without transposing the image. Using this implementation, I removed `image_orig.shape[1] // 4` horizontal seams from *sea.jpg*. The horizontally reduced result is shown in the figure above and serves as the final output for this experiment.



Fig. 6 The horizontal reduced result

On this image, the algorithm correctly chooses low-energy regions in the sky and open water, so most seams pass above the shoreline and around the rocky edges. The visual effect is that the horizon becomes slightly wavy near the center, delicate foam bands are compressed or broken, and the waterfall appears shortened where seams intersect its upper region. These are characteristic artifacts of horizontal carving: as many meandering paths are removed, long, nearly horizontal structures accumulate local bending and nonuniform compression. The result image also reveals a second consequence of selecting horizontal seams on this scene. Because the sky is largely low-energy, it is carved first and thus almost disappears after a quarter-height reduction.

This preserves the rocky coastline and shoreline contours effectively, but the trade-off is a loss of background context and depth that the sky provided, along with a greater risk of subtle horizon curvature.

Horizontal reduction is preferred when the goal is to decrease height while protecting vertical structures and salient boundaries that would be distorted by vertical carving. It is advantageous when important content aligns vertically—such as people, trees, buildings, or cliffs—and there exist horizontally extended low-energy regions, for example large bands of sky, water, road, or grass that can be shortened with minimal perceptual cost. In the present case, choosing horizontal seams successfully concentrates changes in the sky and open water and largely preserves the coastline; however, as the result image demonstrates, aggressive height reduction inevitably trades away much of the sky and can introduce minor rippling along the horizon.

III.

Besides image resizing, Seam Carving algorithm can be used for other applications as mentioned in [1]. Two such applications are content amplification and object removal. Examples of these two applications are shown below. Try to implement these two applications with provided starter codes “seamCarvingContentAmplification.py” and “seamCarvingObjectRemove.py”.

Ans:

In the provided code, content amplification is implemented by first expanding the image with a conventional geometric resize and then applying seam carving reduction to return to the original resolution while preferentially removing pixels from low-energy regions. The routine `seamCarvingContentAmplification` upsamples the input (using standard resizing) to a larger canvas and computes an energy map with `calcEnergy`. It then invokes `findOptSeam` and `reduceImageByIndexArray` repeatedly to delete the required number of seams so that the final output size matches the original. Because seams avoid high-gradient structures, salient elements occupy a larger fraction of the frame after the reduction phase. The result in Fig. 7 reflects this pipeline: compared with the original framing, the coastline, waterfall, and shoreline retain their geometry while the low-texture water and sky contribute most of the removed pixels, making the subject appear proportionally larger without uniform stretching.

Object removal is realized by steering seams through a user-specified mask. In `seamCarvingObjectRemove`, an energy map is computed and the values inside the mask are set to negative infinity before calling `findOptSeam`, guaranteeing that the optimal path traverses the masked region. The algorithm then removes that seam with `reduceImageByIndexArray` and correspondingly shrinks the mask with

reduceMaskByIndexArray. This loop continues until the mask has been fully carved away. In this project implementation, there is no reinsertion step to restore the original width; the output is therefore narrower by the number of removed seams. The outcome in Fig. 8 shows the masked object eliminated and the surrounding background pulled together along low-energy corridors, with the overall width reduced as dictated by the number of iterations.



Fig. 7 The result of content amplification



Fig. 8 The result of object removal (Vertical)

Together, these examples demonstrate how the same core primitives—`calcEnergy` for gradient-based saliency, `findOptSeam` for dynamic programming over admissible paths, and the reduce-by-index operations for pixel deletion—are composed differently to achieve distinct goals. Content amplification enlarges the canvas first and then uses seam removal to give more visual real estate to important structures while preserving the original resolution, as seen in Fig. 7. Object removal directs seams through a specified region and terminates once that region is excised, yielding a coherent fill from neighboring content but a smaller width in the final image, as illustrated in Fig. 8.

IV.

In the object removal part, what is the difference between removing vertical seams and removing horizontal seams. Which one removes object with less seams? Provide one example with images to explain your idea.

Ans:

For object removal, the number of seams required is governed by the object's span along the removal direction: vertical carving must delete approximately as many seams as the object's width in pixels, while horizontal carving needs about the object's height. In the present scene, the target—the small offshore rock circled in Fig. 9—is noticeably wider than it is tall, so a horizontal strategy removes the mask with fewer seams than a vertical one.

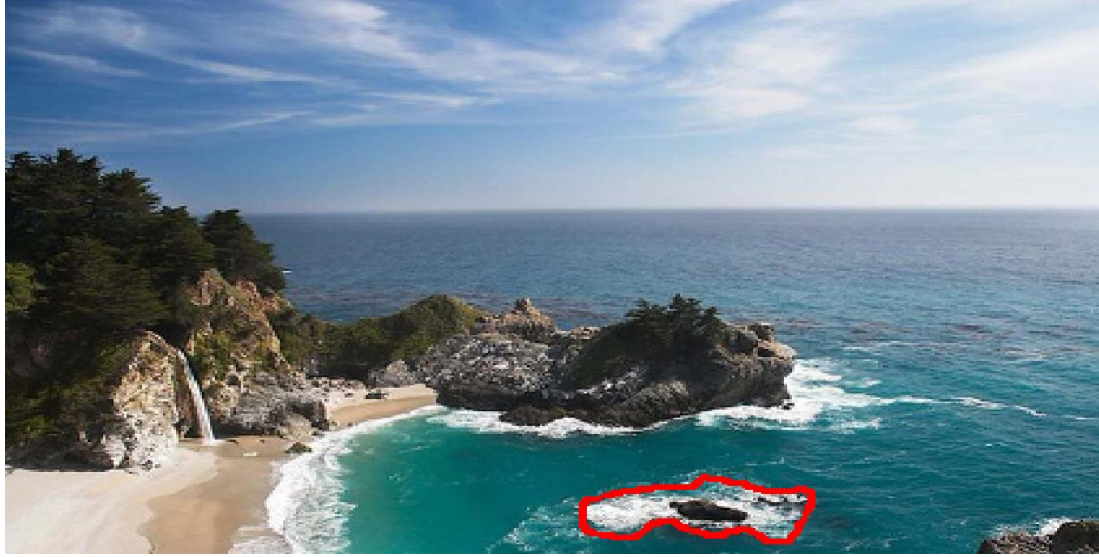


Fig. 9 The zone be circled to form the mask



Fig. 10 The result of object removal (horizontal)

Despite this numerical advantage, the surrounding image structure strongly influences visual quality. Horizontal carving in this image must repeatedly cross the global, high-contrast horizon and the long foam bands, which forces many optimal paths to tilt and merge along that boundary. The resulting accumulation of meandering deletions produces a shear-like artifact and a slanted horizon, as visible in Fig. 10. By contrast, the vertical version previously reported (Fig. 8 in the earlier section) routes seams predominantly through the low-texture water columns. Although it requires more

seams because the object is wide, it preserves the straightness of the horizon and maintains better continuity along the beach and cliff edges.

In this case, horizontal seams would eliminate the rock with fewer iterations, but vertical seams are preferred because they produce a more coherent fill and avoid the diagonal tearing seen in Fig. 10. The example underscores that seam count and visual plausibility can diverge: direction should be chosen by balancing the object's aspect ratio against the global orientation of strong edges, with priority given to the direction that least disrupts dominant structures such as the horizon.

5. Conclusion

This lab demonstrated that seam carving enables content-aware resizing by coupling a gradient-based energy map with dynamic programming to select admissible seams for removal or insertion. In the in-class pipeline, repeated removal yields faithful reductions that preferentially modify low-energy regions, while enlargement via pre-recorded seams with interpolation preserves boundary continuity and suppresses banding. Experiments confirmed that directly inserting the current minimum-energy seam degenerates toward column replication when doubling width and introduces visible artifacts, whereas the demo method distributes seams and blends intensities for superior visual quality.

Extending beyond resizing, the same primitives support content amplification and object removal. Amplification enlarges the canvas and then carves away low-importance pixels so salient structures occupy more of the frame without global stretching. Object removal biases energy through a user mask so seams excise the target and pull background content together; in our implementation the image width decreases by the number of removed seams. Directionality matters: horizontal carving favors scenes with expendable horizontal bands but risks bending long horizons, while vertical carving better preserves such global structures even if more seams are required. Across tasks, results highlight a consistent trade-off between seam count and perceptual plausibility, suggesting that orientation, energy design, and interpolation strategy should be chosen to protect dominant edges and textures in each scene.

6. Reference

- [1] Department of Electrical Engineering, National Tsing Hua University, *EE3662-2025-Unit II-Visual Signal*.
- [2] Department of Electrical Engineering, National Tsing Hua University, *Lab8*.